

Agentic AI: A Survival Kit For Multimedia Content Recommendation

Konstantinos Theodoropoulos
ICE Department
University of West Attica
0009-0001-1233-3019

Georgios Drakopoulos
ICE Department
University of West Attica
0000-0002-0975-1877

Georgios Bardis
ICE Department
University of West Attica
gbardis@uniwa.gr

Phivos Mylonas
ICE Department
University of West Attica
0000-0002-6916-3129

Abstract—Agentic AI has found a plethora of applications in the area of multimedia including content management, content recommendation, and role classification for multimedia entities. Although the particular roles depend heavily on the context and the underlying application, common roles pertain to content categories with successful categorization boosting recommendation. AI agents driven by LLMs rely on the latter not only for receiving responses in natural language, but also for choosing the proper tool for performing tasks if necessary. In the context of content categorization, one such tool can be a graph neural network (GNN) performing vertex classification based on connections and attributes as appropriate on vertices representing multimedia content. Here is presented the architecture of an AI agent with function calling to perform vertex classification on a benchmark graph.

Index Terms—Graph neural networks; GNN; agentic AI; Re-Act model; reasoning; large language models; LLM; higher order attributes; recommendation; multimedia content.

1. Introduction

Role classification [1] and role mining [2] in general when applied to multimedia content can vastly improve the recommendation algorithms for the latter. For instance, categorization of new or even of user generated content based on one or more similarity metrics can ensure that the proper demographics can access it. As another concrete example, when a new account joins a social medium, then it can get accurate recommendations from a huge pool of similar accounts which may be also relatively new in addition to recommendations from a much smaller pool of big accounts and influencers. Then, these two recommendation lists can be fused with either sophisticated algorithm techniques or just be presented as two separate lists to the new account to accelerate its online interaction and also make it feel more welcome to the vast world of social media.

In view of how important recommendation and engagement algorithms are across a broad spectrum not only of online social networks in particular, but also of multimedia content providers. With the advent of agentic AI multimedia recommendation got a new boost as the large language model (LLM) driving the agent may well create a schedule

for efficiently categorizing multimedia content and then to reason how to utilize available tools in order to perform content classification either as a standalone step or as a preprocessing step in order to increase the efficiency of recommendation algorithms. Outside the field of multimedia personalization, another potential application within startup incubators would be the recommendation of partners or the recommendation of startups to mentors and angel investors, forming thus the basis of a framework for evaluating the innovation potential of heterogeneous enterprise ecosystems.

TABLE 1. NOTATION SYNOPSIS.

Symbol	Meaning	First in
$\triangleq$	Definition or equality by definition	Eq. (1)
$ S $	Set cardinality	Eq. (2)
$\deg(v)$	Vertex degree	Eq. (2)
$E[X]$	Mean value of rv X	Eq. (8)
$\text{Var}[X]$	Variance of rv X	Eq. (8)
$f^{(n)}(x)$	n -th derivative of function f	Eq. (12)
$\text{prob}\{\Omega\}$	Probability of event Ω occurring	Eq. (13)

The primary research objective of this conference paper is the demonstration of how a graph neural network (GNN) trained in PyTorch¹ using an extended set of local ground truth attributes can be used in the context of function calling from an AI agent. To this end, the agent architecture is laid out in detail. Said set includes first, second, and higher order attributes such as the vertex degree, the shared neighborhood size, and the cumulants of the degree distribution respectively. To illustrate the feasibility of the proposed approach, the agentic architecture has been applied to a synthetic benchmark graph pertaining to multimedia content.

The structure of this work follows. Section 2 briefly summarizes the relevant scientific literature. In section 3 the agentic AI architecture is described. The synthetic graph and the results are the focus of section 4. Finally, future research directions are given in section 5. The terms *attribute* and *feature* are used interchangeably. Technical acronyms are explained the first time they are encountered in the text. Finally the notation of this work is summarized in table 1.

1. <https://pytorch.org>

2. Related Work

Multimedia recommendation [3] is a very diverse field [4][5] with a wide array of applications such as cultural content recommendation [6], tensor completion based clustering for movies [7], adaptive clustering for multimedia preferences [8], asymptotics aware recommendation for content personalization [9], and behavioral enhanced recommendation [10]. Higher order attributes such as cumulants [11][12] and the Laplacian spectrum [13] reveal more information about a distribution compared to first order statistics [14][15] and can be combined with the above schemes. Tensor driven community discovery can be used for recommendation [16] in order to accomodate multilinear attribute sets.

Agentic AI [17][18] has already found numerous applications since the introduction of large language models (LLMs) [19] and vector databases [20]. Extractive document summarization [21] can enhance the training and performance of agents [22][23]. Set cardinality estimators [24] can efficiently guide LLMs to estimate their workload [25] and to generate succinct responses in natural language [26].

GNNs have been used in conjunction with higher order attributes [27] to achieve better performance [28] for a graphs like heterophilic [29] and heterogeneous [30]. GNNs can be combined with transformers [31] and edge splitting [32], while resource allocation can accelerate learning [33]. Applications include multiview spectral graph clustering [34], topological graph clustering [35][36][37][38], mining network structure [39][40], zero-shot learning [41], localizing software faults [42], wireless communications [43], and partial differential equations (PDEs) [44]. Graph data structures can help a GNN reconfigure itself and achieve high performance in the face of evolving topologies [45].

3. Architecture

3.1. Agentic AI

Agentic AI relies heavily on *function calling* or *tool calling*, namely the passing to one or more of the available functions the proper arguments in order to obtain a result if the LLM driving the agent reasons that there is insufficient information to provide the user with a complete answer. Function calling may also well take place in intermediate steps of the ongoing conversation with the user.

TABLE 2. AGENTIC AI VS HUMAN CODE.

Parameter	Agentic AI	Human code
Path	Dynamic through reasoning	Static with fixed points
Failure	Adapts and responds	Throws exception
Operation	Rule-dependent	Task-dependent
Interaction	Iterative and interactive	Passive

The main loop of any agentic AI is essentially the iterative execution cycle of the ReAct model². This model is

shown in figure 1. Moreover, the differences between agentic AI and human generated code are listed in table 2.

The main components in an agentic AI architecture as well as their respective roles are delineated below. Although the latter may slightly vary across architectures, the general guidelines still apply.

- **LLM:** A language model that takes text as input and produces text as output. Can be prompted with questions by the user, system prompts, and also with tool descriptions.
- **Agent:** A piece of software acting on behalf of the user charged with executing instructions issued by the LLM, primarily through tools, and returning their results to it.
- **Tool:** A dedicated autonomous code segment, whether a function or an entire application, which carries out specific computations with arguments supplied by the LLM through the agent.

The LLM is driven mainly by two kinds of prompt with different roles and different priorities which determine the starting point and the limitations of its reasoning. These are the following:

- **System prompt:** It should contain the description of each tool, possibly as a JSON object, along with a description and the type of each argument a given tool expects.
- **User prompt:** The interactions with the user are coded as a sequence of messages to the LLM. These contain the instructions and the requests of the user within the limits of the system prompt.

The sequence of user prompts is kept, especially when it is short, in order to provide much needed context for the LLM. Alternatively, it must be stored in special state variables representing an aggregation of the conversation on many levels like linguistic, emotional, semantic, and pragmatologic. Moreover, important terms and references may be separately stored. The sequence of user prompts is essential as different sequences may well lead to different context depending on the implementation. Moreover, parts of the user prompt may be dropped if they violate the guidelines set forth in the system prompt.

To illustrate the above, consider the following simplified example. Therein there is a single tool called `degree` which returns the number of neighbors of a given vertex or raises a `ValueError` exception when a `None` placeholder value is passed. When the list is empty, then zero is returned.

```
def degree(vertices: list | None) -> int:
    if vertices is None:
        raise ValueError('No list.')
    return len(vertices)
```

The tool description for function calling is a Python list of dictionaries where each dictionary contains a short description of a given tool, which can be used by the LLM not only as a guide to determine whether the corresponding tool is appropriate for a given task but also to justify to

2. <https://www.ibm.com/think/react-agent>

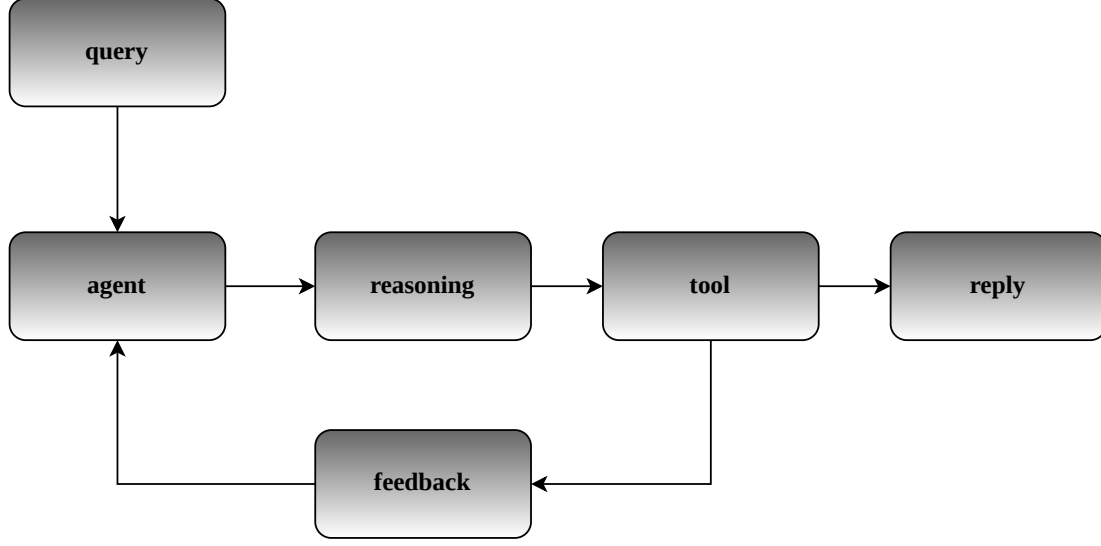


Figure 1. The ReAct model conceptual loop.

the user in natural language why this particular tool was chosen if so requested. Moreover, in the same dictionary are stored the arguments the tool requires, a brief description of them, their type, and whether they are required. This allows the agent to properly formulate and pass the arguments to the tool based on the choices by the user and any relevant guidelines found in the system prompt.

```

tools = [
  {
    "name": "degree",
    "function": "Compute vertex degree",
    "parameters": {
      "type": "object",
      "properties": {
        "vertices": {"type": "list"}
      },
      "required": ["vertices"]
    }
  }
]

```

The above components serve to make agentic AI architectures reliable, efficient, and capable of interacting with humans in natural language, a major breakthrough from a technological as well as from a UI/UX perspective.

3.2. PyTorch as a tool

In this context the GNN is the single tool at the disposal of the agent. Although GNNs can and often have been standalone tools, nothing prevents them from being a component in a function calling architecture with multiple tools for achieving a broader objective. The GNN utilized as a tool by the agent is trained synchronously in an iteratively manner as shown in algorithm 1 to learn to classify its vertices to two categories, namely as being relevant or not to the user. This

serves as a major early pipeline step for more sophisticated recommendation schemes.

Algorithm 1 GNN training

Require: Graph and ground truth vectors.

Require: Termination condition set.

Ensure: Graph is trained.

```

1: while termination conditions are false do
2:   determine dropout set  $S$  for current round
3:   for all vertices  $u$  not in  $S$  do
4:     exchange messages with neighbors
5:     update local ground truth vector
6:   end for
7: end while
8: return trained graph

```

The dropout set S during each round reduces the computational cost per round and additionally it smoothes out the training process speed among vertices as in its absence high degree vertices converge or, for pathological cases of course, reach problematic states faster. When a vertex is trained intensely compared to the other ones, then it may converge quicker, but this convergence state may very well be unstable and thus pushing the vertex past that point through excessive training may create ripples throughout the graph.

The attributes used in the training of the GNN are of various orders. Higher order in the context of graph mining refers to the number of vertices necessary to construct an attribute about a given vertex u . Therefore, a first order feature involves information only about u , a second order attribute requires information about another vertex, most likely the neighbors of u , whereas a higher order one entails the examination of at least two distinct vertices besides u , for instance the neighbors or the neighbors of u . Hence, under this definition, transforms whether linear or nonlinear

of first and second order features do not count as higher order attributes.

The features used to train the GNN are shown in table 3. In the leftmost column the local attributes, namely the first and second order features, are listed, then in the middle the local higher order attributes are given, and finally on the rightmost column the global graph attributes are stated. The difference between the middle and the right column serves to show to the GNN how the profile of a given vertex differs from that of a typical vertex, whereas the right column adds local context in terms of variation and details.

One of the core local attributes of the local ground truth feature set of a vertex u is the vector $\mathbf{d}_u \in \mathbb{Z}^{n_u}$ consisting of the degrees of u and these of its neighbors as shown in equation (1). This reveals details of the neighborhood.

$$\mathbf{d}_u \triangleq [\deg(v), \deg(s), \dots, \deg(c)]^T \quad (1)$$

In equation (1) the length n_u of the vector $\mathbf{d}_u$ by construction equals the degree of u as it ranges over the neighborhood $\Gamma(u)$ of u as shown in equation (2).

$$n_u \triangleq |\Gamma(u)| = \deg(u) \quad (2)$$

An important higher order attribute for vertex u is the clustering coefficient $\rho(u)$ defined as the ratio of the closed triangles containing u to the potential number of triangles containing u . Collecting the clustering coefficients of u and these of its neighbors to a vector $\mathbf{r}_u$ as shown in equation (3) yields important local structural information about u .

$$\mathbf{r}_u \triangleq [\rho(u), \rho(v), \dots, \rho(s)]^T, v, s \in \Gamma(u) \quad (3)$$

From an algorithmic engineering perspective, the more transformed features are available, typically the more efficient the graph learning process is as GNN communication and feedback loops benefit from high attribute variability. To this end, a number of nonlinear transforms have been applied to both $\mathbf{r}_u$ and $\mathbf{d}_u$. The three Pythagorean means, namely the arithmetic, the geometric, and the harmonic means described in equations (4), (5), and (6) respectively provide ways to do so. The arithmetic mean is easy and efficient to compute as shown in equation (4) but it is prone to outliers.

$$\mathcal{A}(\mathbf{x}) \triangleq \frac{1}{n} \sum_{k=1}^n \mathbf{x}[k] \quad (4)$$

The geometric mean for a numerical vector $\mathbf{x}$ defined as in equation (5). Although it is much less prone to outliers like the arithmetic mean, the numerical stability of both forms may be challenging under certain circumstances.

$$\mathcal{G}(\mathbf{x}) \triangleq \left(\prod_{k=1}^n \mathbf{x}[k] \right)^{\frac{1}{n}} = \exp \left(\frac{1}{n} \sum_{k=1}^n \ln \mathbf{x}[k] \right) \quad (5)$$

The harmonic mean for a numerical vector $\mathbf{x}$ is defined as in equation (6) shown below. Regarding numerical stability, small entries in $\mathbf{x}[k]$ may lead to catastrophic cancellation and a small denominator might yield a highly inaccurate value. Nonetheless, the harmonic mean is much less prone

to outliers than both the arithmetic and geometric means. The harmonic mean is used among others in information retrieval (IR) as well as in computer network analysis.

$$\mathcal{H}(\mathbf{x}) \triangleq \frac{n}{\sum_{k=1}^n \frac{1}{\mathbf{x}[k]}} \approx \frac{n}{\sum_{k=1}^n \frac{1}{\max\{\mathbf{x}[k], x_0\}}} \quad (6)$$

The relationship between the three Pythagorean means above indicates that there is attribute variability.

$$\mathcal{H}(\mathbf{x}) \leq \mathcal{G}(\mathbf{x}) \leq \mathcal{A}(\mathbf{x}) \quad (7)$$

Another set of nonlinear transforms consists of the skewness and kurtosis shown in equations (8) and (9) respectively. They are third and fourth order cumulants characterizing the form of a distribution. Specifically, the skewness of an rv $\mathcal{X}$ is a measure of how symmetric its distribution is around its mean depending on its sign. This single metric can be applied to normalized versions of $\mathbf{d}_u$ and $\mathbf{r}_u$ which can be treated as probability mass functions.

$$\mathbb{E} \left[\left(\frac{\mathcal{X} - \mathbb{E}[\mathcal{X}]}{\sqrt{\text{Var}[\mathcal{X}]}} \right)^3 \right] = \frac{\mathbb{E}[\mathcal{X} - \mathbb{E}[\mathcal{X}]]^3}{\text{Var}[\mathcal{X}]^{\frac{3}{2}}} \quad (8)$$

The kurtosis shown in equation (9) is a metric showing how heavy the tail of a distribution is. This metric can more accurately capture how heavy a distribution is compared to probabilistic inequalities like the Chebyshev inequality.

$$\mathbb{E} \left[\left(\frac{\mathcal{X} - \mathbb{E}[\mathcal{X}]}{\sqrt{\text{Var}[\mathcal{X}]}} \right)^4 \right] = \frac{\mathbb{E}[\mathcal{X} - \mathbb{E}[\mathcal{X}]]^4}{\text{Var}[\mathcal{X}]^2} \quad (9)$$

The characteristic function $\Psi(j\omega; \mathcal{X})$ of a rv $\mathcal{X}$ is defined as the conjugated Fourier transform of its probability distribution as shown in equation (10). It can reveal important information about the distribution through its form, its derivatives, and other additional properties like its convergence region(s). Also the Taylor expansion of the characteristic function can add significant information.

$$\Psi(j\omega; \mathcal{X}) \triangleq \mathbb{E}[e^{j\omega\mathcal{X}}] = \sum_{k=1}^n p_k e^{j\omega(k-1)} \quad (10)$$

Observe that (10) is by construction a trigonometric polynomial of degree $n-1$ with real coefficients, where n is the number of events in the distribution. An important property regularization property is shown in equation (11).

$$\Psi(0; \mathcal{X}) = \sum_{k=0}^{n-1} p_k = 1 \quad (11)$$

Another way to describe $\Psi(j\omega; \mathcal{X})$ is through its cumulants c_k , which are the Taylor coefficients of the logarithm of $\Psi(j\omega; \mathcal{X})$ evaluated at zero. These alternative coefficients also reveal information about the shape of the distribution and have numerous applications in signal processing, including audio processing and graph signal processing.

$$c_k \triangleq \frac{\partial^k \ln \Psi(j\omega; \mathcal{X})}{\partial (j\omega)^k} \Big|_{\omega=0} \quad (12)$$

TABLE 3. VERTEX ATTRIBUTES IN THE GROUND TRUTH VECTORS.

Lower order attributes	Higher order attributes	Global attributes
Neighborhood size (degree)	Clustering coefficient of	Global clustering coefficient
Local degree vector	Pythagorean means	Graph diameter
Common neighbors	Adamic-Adar index	Number of triangles

The above contain only a small subset of the ground truth vectors embedded in every vertex. The same holds true for the entries of table 3.

4. Results

In this section the benchmark graph used in described and the results obtained from it are explained. The graph comes from a cultural game designed to teach and promote a set of values enshrined in classical antiquity. The vertices represent areas of the game to be recommended to the player, where edges represent the connections between them.

TABLE 4. BENCHMARK GRAPH PROPERTIES.

Property	Value
Number of vertices	142
Number of edges	371
Triangles	516
Squares	331
Diameter	8
Attributes per vertex	97

Regarding the recommendation part, as the graph is annotated with the actual choices of players, the accuracy of the GNN can be computed and compared against the ground truth data and it was found to be 91.33%, which is rather encouraging although there is clearly room for improvement.

Concerning the function calling part, the ability of the LLM to understand whether it should call its tool without much explicit user guidance is evaluated by the right call rate, which for a single tool τ it takes the form shown in equation (13) below. In our case ρ was 98.17%.

$$\rho \triangleq \frac{\tau \text{ was called}}{\tau \text{ should have been called}} \approx \text{prob} \{ \tau \text{ was correctly called.} \} \quad (13)$$

When multiple tools are available, then ρ can be parameterized depending on whether a given tool should have been called, leading to finer granularity results.

5. Conclusions

This conference paper focuses on the potential of agentic AI and function calling for graph mining. In particular, an agent has the ability to call PyTorch to train a graph neural network (GNN) to classify the vertices of a network whose vertices represent multimedia content with the purpose of selecting which vertices should be recommended to the user. Said GNN is trained with local ground truth vectors containing lower and higher order structural patterns. The latter offer a considerable algorithmic advantage in terms of

accuracy. GNNs have been selected as they offer superior performance in classification problems by taking advantage of the knowledge inherent in industrial processes which typically rely on maintaining and, should the need arise, re-configuring connectivity patterns between its various paths.

This work can be extended in a number of ways. First and foremost, more and attributes can be used to augment the benchmark graph and also more graphs representing pieces of multimedia content and the interconnections between them can be used. Additionally, regarding the recommendation part, the effect of the GNN classification to reranking should be evaluated. About the function calling part, the agent can be given access to preconfigured GNN classifiers or the ability to create new ones by passing detailed information to PyTorch. Additionally, the agent can be given the ability to combine the output of different GNN classifiers or even results coming from other types of vertex classifiers. Finally, numerical stability considerations can be explored, given the size of multimedia content graphs.

Acknowledgment

This conference paper was written as part of Project 451, a long term research initiative whose primary objective is the development of novel, scalable, numerically stable, and interpretable higher order analytics.

References

- [1] L. Zhu, C. Zheng, W. Guan, J. Li, Y. Yang, and H. T. Shen, "Multi-modal hashing for efficient multimedia retrieval: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 239–260, 2023.
- [2] D. Ghai, S. Saxena, G. Dhingra, and S. L. Tripathi, "A comprehensive review on performance-based comparative analysis, categorization, classification and mapping of text extraction system techniques for images," *Multimedia Tools and Applications*, vol. 84, no. 5, pp. 2327–2484, 2025.
- [3] L. Wu, Y. Yang, K. Zhang, R. Hong, Y. Fu, and M. Wang, "Joint item recommendation and attribute inference: An adaptive graph convolutional network approach," in *SIGIR*. ACM, 2020, pp. 679–688.
- [4] S. Sattari and A. Yazici, "Semantic deep learning and adaptive clustering for handling multimodal multimedia information retrieval," *Multimedia Tools and Applications*, vol. 84, no. 13, pp. 11 795–11 831, 2025.
- [5] Y. Gan, Y. You, J. Huang, S. Xiang, C. Tang, W. Hu, and S. An, "Multi-view clustering via multi-stage fusion," *IEEE Transactions on Multimedia*, vol. 27, pp. 4571–4583, 2025.
- [6] G. Drakopoulos, I. Giannoukou, S. Sioutas, and P. Mylonas, "Self organizing maps for cultural content delivery," *NCAA*, vol. 31, no. 7, 2022.

- [7] H. Wang, Y. Chen, M. Yao, W. Liu, J. Peng, and X. Fu, "Tensor completion framework by graph refinement for incomplete multi-view clustering," *IEEE Transactions on Multimedia*, 2025.
- [8] M. Xiong, K. Hu, Z. Wang, R. Hu, K. Muhammad, J. Del Ser, X. Yang, and B. Sheng, "Adaptive clustering and weighted regularization contrastive learning framework for unsupervised person re-identification," *IEEE Transactions on Multimedia*, vol. 27, pp. 6896–6907, 2025.
- [9] Y. Xu, S. Chen, J. Li, and J. Yang, "Asymptotics-aware multi-view subspace clustering," *IEEE Transactions on Multimedia*, vol. 27, pp. 3650–3663, 2025.
- [10] G. Drakopoulos, Y. Voutos, P. Mylonas, and S. Sioutas, "Motivating item annotations in cultural portals with UI/UX based on behavioral economics," in *IISA*. IEEE, 2021.
- [11] M. Pakravan and M. B. Shamsollahi, "Extraction and automatic grouping of joint and individual sources in multisubject fMRI data using higher order cumulants," *IEEE Journal of biomedical and health informatics*, vol. 23, no. 2, pp. 744–757, 2018.
- [12] P. Bonnier, H. Oberhauser, and Z. Szabó, "Kernelized cumulants: Beyond kernel mean embeddings," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [13] Y. Zhuo, S. Zhou, and L. Yang, "Laplacian spectrum of two classes of ψ -sum graphs with applications," *Circuits, Systems, and Signal Processing*, pp. 1–24, 2024.
- [14] H. Cho, M. Jeong, S. Jeon, S. Ahn, and W. H. Kim, "Multi-resolution spectral coherence for graph generation with score-based diffusion," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [15] C. Li, J. Fu, Y. Yan, Z. Zhao, and Q. Zeng, "Higher order heterogeneous graph neural network based on node attribute enhancement," *Expert Systems with Applications*, vol. 238, 2024.
- [16] G. Drakopoulos, K. C. Giotopoulos, I. Giannoukou, and S. Sioutas, "Unsupervised discovery of semantically aware communities with tensor Kruskal decomposition: A case study in Twitter," in *SMAP*. IEEE, 2020.
- [17] M. Abou Ali, F. Dornaika, and J. Charafeddine, "Agentic AI: A comprehensive survey of architectures, applications, and future directions," *Artificial Intelligence Review*, vol. 59, no. 1, p. 11, 2025.
- [18] A. K. Pati, "Agentic AI: A comprehensive survey of technologies, applications, and societal implications," *IEEE Access*, vol. 13, pp. 151 824–151 837, 2025.
- [19] D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous intelligence for complex goals—A comprehensive survey," *IEEE Access*, vol. 13, pp. 18 912–18 936, 2025.
- [20] G. Drakopoulos, L. Theodorakopoulos, S. Sioutas, and P. Mylonas, "Functional programming meets Pinecone: Recommending graph structured documents," in *IEEE Big Data*. IEEE, 2025.
- [21] E. Arvanitis, G. Drakopoulos, L. Theodorakopoulos, S. Sioutas, and P. Mylonas, "Extractive document summarization with graph neural networks and topic modeling in PyTorch," in *IEEE Big Data*. IEEE, 2025.
- [22] S. Murugesan, "The rise of agentic AI: Implications, concerns, and the path forward," *IEEE Intelligent Systems*, vol. 40, no. 2, pp. 8–14, 2025.
- [23] T. Raheem and G. Hossain, "Agentic AI systems: Opportunities, challenges, and trustworthiness," in *IEEE International Conference on Electro Information Technology (eIT)*. IEEE, 2025, pp. 618–624.
- [24] G. Drakopoulos, S. Kontopoulos, and C. Makris, "Eventually consistent cardinality estimation with applications in biodata mining," in *SAC*. ACM, 2016.
- [25] F. Jiang, C. Pan, K. Wang, P. Michiardi, O. A. Dobre, and M. Debbah, "From large AI models to agentic AI: A tutorial on future intelligent communications," *IEEE Journal on Selected Areas in Communications*, 2026.
- [26] G. R. Gonzalez, J. Habel, and G. K. Hunter, "AI agents, agentic AI, and the future of sales," *Journal of Business Research*, vol. 202, 2026.
- [27] J. Li, H. Peng, Y. Cao, Y. Dou, H. Zhang, S. Y. Philip, and L. He, "Higher-order attribute-enhancing heterogeneous graph neural networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 560–574, 2021.
- [28] J. You, J. Leskovec, K. He, and S. Xie, "Graph structure of neural networks," in *ICML*. PMLR, 2020, pp. 10 881–10 891.
- [29] W. Bi, L. Du, Q. Fu, Y. Wang, S. Han, and D. Zhang, "Make heterophilic graphs better fit GNN: A graph rewiring approach," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [30] S. Wang, G. Cao, W. Cao, and Y. Li, "NLA-GNN: Non-local information aggregated graph neural network for heterogeneous graph embedding," *Pattern Recognition*, 2024.
- [31] Y. Sun, D. Zhu, Y. Wang, Y. Fu, and Z. Tian, "GTC: GNN-Transformer co-contrastive learning for self-supervised heterogeneous graph representation," *Neural Networks*, 2024.
- [32] J. Guo, K. Huang, R. Zhang, and X. Yi, "ES-GNN: Generalizing graph neural networks beyond homophily with edge splitting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [33] Y. Peng, J. Guo, and C. Yang, "Learning resource allocation policy: Vertex-GNN or edge-GNN?" *IEEE Transactions on Machine Learning in Communications and Networking*, 2024.
- [34] Z. Xie, C. Jiao, K. Wu, D. Guo, and R. Hong, "Active factor graph network for group activity recognition," *IEEE Transactions on Image Processing*, 2024.
- [35] G. Drakopoulos and P. Mylonas, "Clustering MBTI personalities with graph filters and self organizing maps over Pinecone," in *Big Data*. IEEE, 2024.
- [36] K. Ullah, H. Garg, T. Mahmood, N. Jan, and Z. Ali, "Correlation coefficients for T-spherical fuzzy sets and their applications in clustering and multi-attribute decision making," *Soft Computing*, vol. 24, pp. 1647–1659, 2020.
- [37] L. Ding, C. Li, D. Jin, and S. Ding, "Survey of spectral clustering based on graph theory," *Pattern Recognition*, 2024.
- [38] J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec, "Identity-aware graph neural networks," in *AAAI*, vol. 35, no. 12, 2021, pp. 10 737–10 745.
- [39] L. Hu, J. Zhang, X. Pan, H. Yan, and Z.-H. You, "HiSCF: Leveraging higher-order structures for clustering analysis in biological networks," *Bioinformatics*, vol. 37, no. 4, pp. 542–550, 2021.
- [40] G. Drakopoulos and P. Mylonas, "A genetic algorithm for Boolean semiring matrix factorization with applications to graph mining," in *Big Data*. IEEE, 2022.
- [41] L. Liu, T. Zhou, G. Long, J. Jiang, and C. Zhang, "Attribute propagation network for graph zero-shot learning," in *AAAI*, vol. 34, no. 04, 2020, pp. 4868–4875.
- [42] X. Gou, A. Zhang, C. Wang, Y. Liu, X. Zhao, and S. Yang, "Software fault localization based on network spectrum and graph neural network," *IEEE Transactions on Reliability*, 2024.
- [43] S. Mourya, P. Reddy, S. Amuru, and K. K. Kuchi, "Spectral temporal graph neural network for massive MIMO CSI prediction," *IEEE Wireless Communications Letters*, 2024.
- [44] R. J. Gladstone, H. Rahmani, V. Suryakumar, H. Meidani, M. D'Elia, and A. Zareei, "Mesh-based GNN surrogates for time-independent PDEs," *Scientific reports*, vol. 14, no. 1, p. 3394, 2024.
- [45] S. Kontopoulos and G. Drakopoulos, "A space efficient scheme for graph representation," in *ICTAI*. IEEE, 2014, pp. 299–303.